

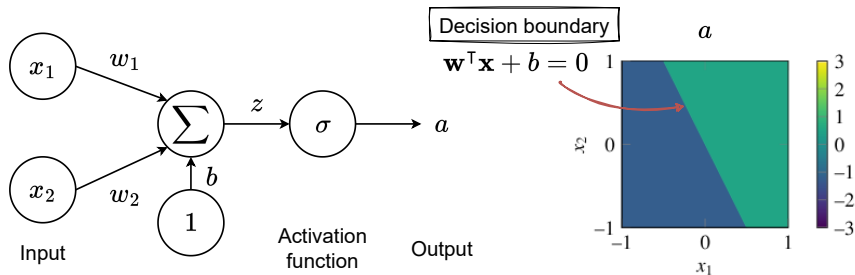
DEEP LEARNING

Neural Networks Part II

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Recap: Perceptron as a Linear Classifier



- Computations in a perceptron:

$$z = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}^T \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + b \rightarrow \text{affine transformation}$$

$$a = \underbrace{\sigma(z)}_{\text{step function}} \rightarrow \text{output}$$

Recap: Perceptron as a Linear Classifier

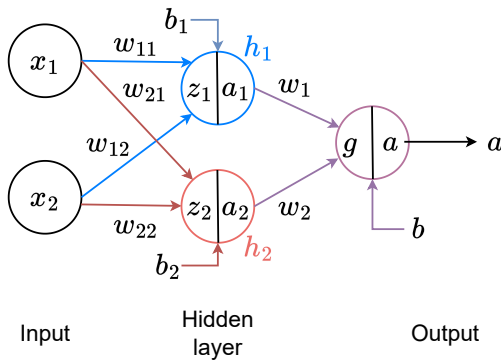
- We can generalize and extend it to d -dimensional input $\mathbf{x} = [x_1, x_2, \dots, x_d]$ as:

$$z = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}^\top \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} + b \quad \rightarrow \text{affine transformation}$$
$$a = \sigma(z) \quad \rightarrow \text{output}$$

- In the original perceptron by Rosenblatt, the activation function was a step function. However, it gradually got replaced by sigmoid activation function.
- Final prediction using a sigmoid activation function is obtained via thresholding:

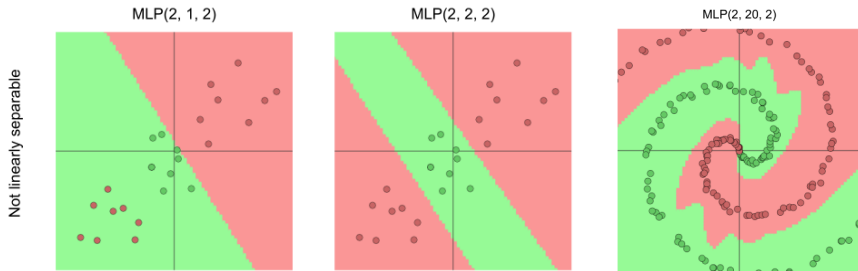
$$\text{Predicted class} = \begin{cases} 1 & \text{if } a \geq 0.5 \\ 0 & \text{if } a < 0.5 \end{cases}$$

Recap: Multilayer Perceptron



- Multilayer Perceptron (or MLP) has at least one hidden layer and the hidden layer has one or more hidden units.
- Each hidden unit is connected to an activation function (eg, sigmoid).
- Each hidden unit learns a linear classifier, which are then combined to obtain a **non-linear classifier** (or non-linear decision boundary).

Recap: MLP as Non-linear Classifier



simple units (each perceptron) + many connections = powerful computation.

Today

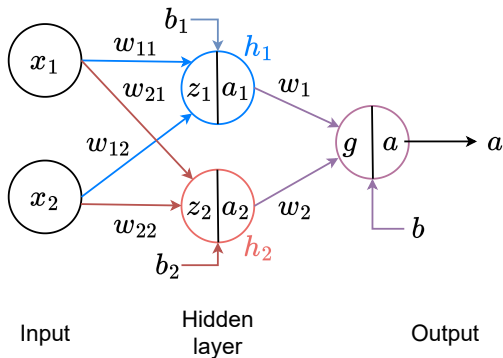
- Vectorization in Multilayer Perceptron
- From binary to multiclass classification
- Understand the **core ingredients** that affect neural network performance:
 - ▶ Activation functions
 - ▶ Weight initialization
 - ▶ Optimization algorithms
 - ▶ Loss functions
 - ▶ Regularization techniques
 - ▶ Normalization techniques

Table of Contents

1. Vectorization
2. Multiclass Classification
3. Activation Functions
4. Weight Initialization
5. Optimization Algorithms
6. Loss Functions
7. Regularization Techniques
8. Normalization Techniques

Vectorization

Vectorization in Multilayer Perceptron



- **Question:** how can we vectorize the affine transformations for different hidden units in an efficient manner?
- **Hint:** recall what we did in linear regression!

Vectorization in Multilayer Perceptron

- Computations at the hidden unit h_1 :

$$z_1 = \begin{pmatrix} w_{11} \\ w_{12} \end{pmatrix}^T \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + b_1, a_1 = \sigma(z_1)$$

- Similarly, computations at the hidden unit h_2 :

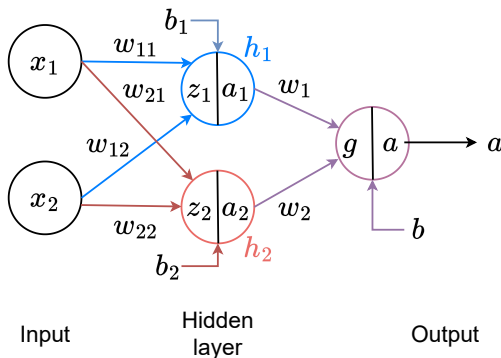
$$z_2 = \begin{pmatrix} w_{21} \\ w_{22} \end{pmatrix}^T \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + b_2, a_2 = \sigma(z_2)$$

- Combining the two dot-products into one matrix-vector product:

$$\mathbf{z} = \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}, \mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} = \sigma(\mathbf{z})$$

- The vector $\mathbf{a}$ is called a **feature vector**.

Vectorization in Multilayer Perceptron

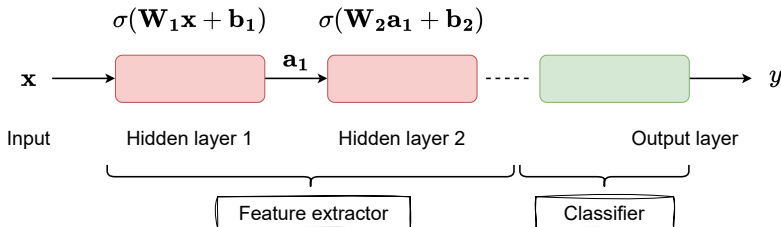


- The vectorization operation at the output layer is similar to the operations at a single hidden unit:

$$g = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}^T \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} + b, a = \sigma(g)$$

- Note that the output is a scalar value.

Vectorization in Multilayer Perceptron



- In the i -th layer of a MLP the following operations take place:
 1. Affine transformation: $\mathbf{z}_i = \mathbf{W}_i\mathbf{a}_{i-1} + \mathbf{b}_i$
 2. Non-linear activation function: $\mathbf{a}_i = \sigma(\mathbf{z}_i)$
- After the final hidden layer, we have an output layer that produces the output of the network y , which is a scalar value for binary classification.
- All the hidden layers are collectively called **feature extractor**, and the output layer is often called the **classifier**.

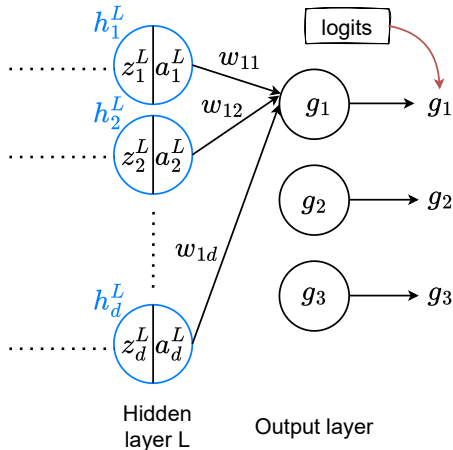
Multiclass Classification

MLP for Multiclass Classification

- In the multiclass classification, the output layer has as many output units (or **logits**) as the number of classes.
- For example, if we are classifying among three animals, we will have three output units.
- The first logit is computed as:

$$g_1 = \begin{pmatrix} w_{11} \\ \vdots \\ w_{1d} \end{pmatrix}^T \begin{pmatrix} a_1^L \\ \vdots \\ a_d^L \end{pmatrix} + b_1^L$$

- Similarly, we compute the other logits: g_2 and g_3 , by using the corresponding weights.

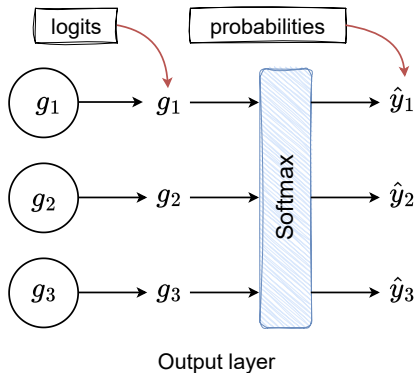


MLP for Multiclass Classification

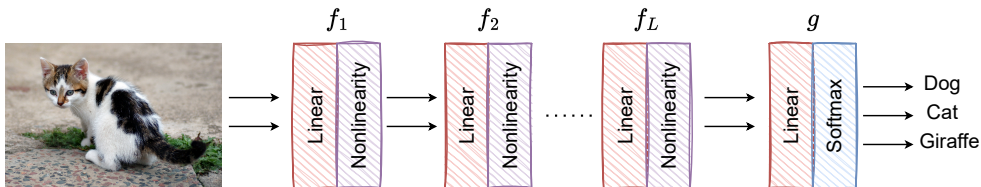
- Following the vectorization technique (as in the hidden layers), we can compute all the logits using one matrix-vector product.
- The logits of the output layer refer to the vector: $\mathbf{g} = [g_1, g_2, g_3]$.
- We apply the softmax function on the logits to get the **output probabilities**, which is again a vector $\hat{\mathbf{y}}$:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{g}),$$
$$\hat{y}_k = \frac{e^{g_k}}{\sum_{i=1}^{K=3} e^{g_i}},$$

where $\hat{y}_k$ is the predicted probability of the k -th class.



Deep Neural Network (DNN)



- DNNs are neural networks that stack the linear-nonlinear motifs many times.
- Each layer is a function. Therefore, a DNN is a composition of many functions:

$$f_{\text{DNN}}(\mathbf{x}) = g(f_L(f_{L-1}(\cdots f_2(f_1(\mathbf{x}))))),$$

where f_i is the function of the i -th layer, and g is the classifier (along with softmax) at the output of the DNN.

- Each layer f_i is associated with some weights $\mathbf{W}_i$ and biases $\mathbf{b}_i$, collectively referred to as parameters. We train or learn the parameters of all layers (including the output layer g) through gradient descent.

Activation Functions

Why Do We Need Activation Functions?

- Neural networks model complex, non-linear relationships.

$$f_{\text{NN}}(\mathbf{x}) = \mathbf{w}^T \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) + b$$

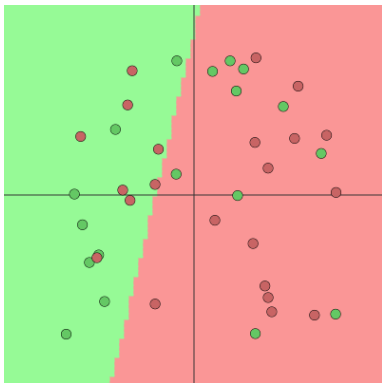
- Without non-linear activations, deep networks collapse into equivalent linear models.

$$\begin{aligned} f_{\text{NN}}(\mathbf{x}) &= \mathbf{w}^T (\mathbf{W}\mathbf{x} + \mathbf{b}) + b \\ &= \underbrace{\mathbf{w}^T \mathbf{W}}_{\hat{\mathbf{w}}^T} \mathbf{x} + \underbrace{\mathbf{w}^T \mathbf{b} + b}_{\hat{b}} = \underbrace{\hat{\mathbf{w}}^T \mathbf{x} + \hat{b}}_{\text{What is this?}} \end{aligned}$$

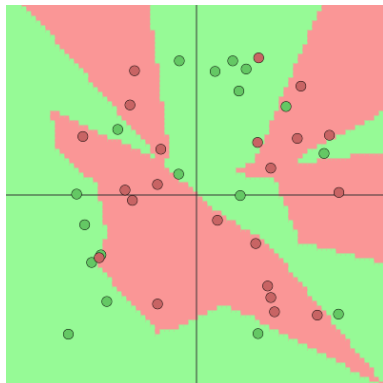
- This is a [single hyperplane](#), i.e., a linear classifier.
- Activation functions introduce non-linearity in the decision boundaries, such that they can be [curved and complex](#).

Why Do We Need Activation Functions?

MLP(2, 10, 5, 2) without activation function



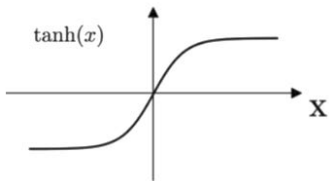
MLP(2, 10, 5, 2) with ReLU activation function



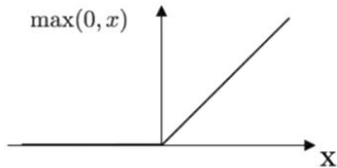
Key takeaway: Activation functions allow networks to learn complex patterns and approximate any function (Universal Approximation Theorem).

Common Activation Functions

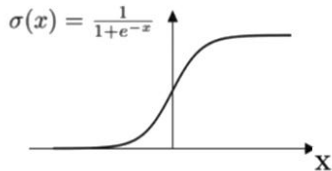
Tanh



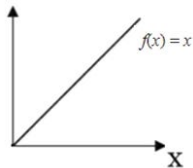
ReLU



Sigmoid

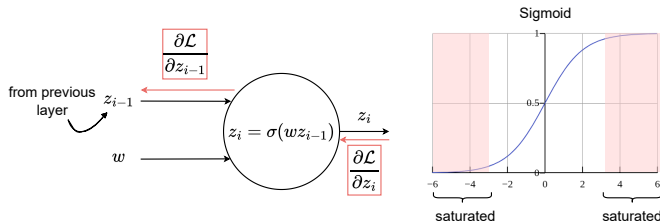


Linear



Vanishing Gradients

- A common problem with sigmoid and tanh activation function is the **vanishing gradients** problem.

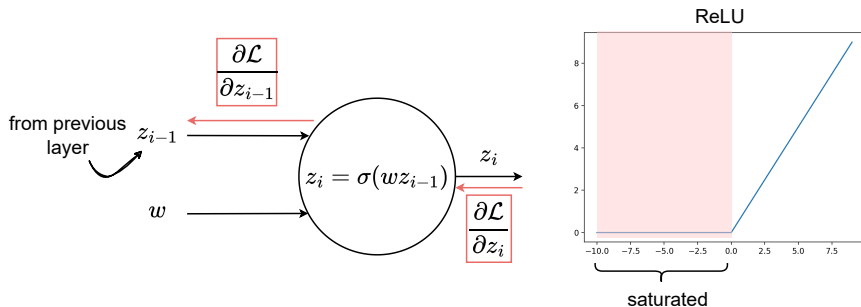


- The gradient backpropagated to the previous layer:

$$\frac{\partial \mathcal{L}}{\partial z_{i-1}} = \frac{\partial \mathcal{L}}{\partial z_i} \frac{\partial z_i}{\partial z_{i-1}} = \frac{\partial \mathcal{L}}{\partial z_i} \frac{\partial}{\partial z_{i-1}} \{\sigma(wz_i)\} = \frac{\partial \mathcal{L}}{\partial z_i} \cdot \underbrace{\sigma'(wz_{i-1})}_{\text{small in saturated regions}} \cdot \frac{\partial}{\partial z_{i-1}} \{wz_i\}$$

- Repeated multiplication of small gradients causes gradients to become smaller (also known as vanishing gradient) in the previous layers.

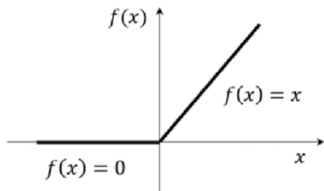
Addressing Vanishing Gradients



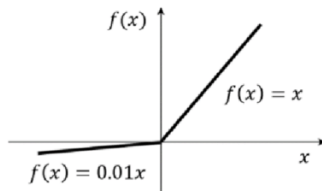
- Vanishing gradient becomes a serious issue with very deep networks.
- ReLU activation function $\max(0, x)$ solves the problem because there is no saturation on the positive side of the function.
- The derivative is a constant, ie, equal to 1, on the positive side.

Addressing Vanishing Gradients

- However, ReLU activation function has no gradient on the negative side of the function!
- To solve this, LeakyReLU is proposed that solves zero gradient on the negative side.



ReLU activation function

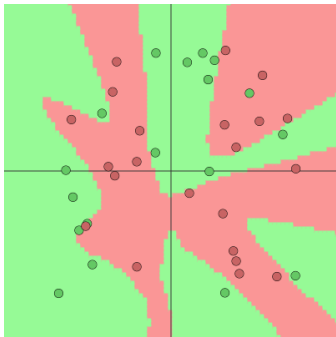


LeakyReLU activation function

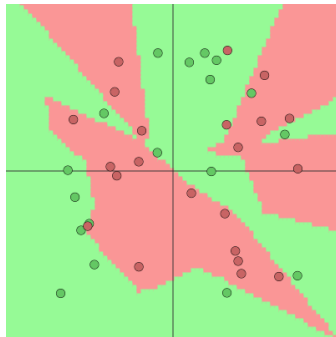
- For most practical cases, ReLU activation functions are enough to avoid vanishing gradient problem.

Impact of Activation Functions

MLP(2, 10, 5, 2) with Sigmoid activation function



MLP(2, 10, 5, 2) with ReLU activation function



- Notice the difference in the decision boundaries.
- Sigmoid leads to **smoother** decision boundaries, but suffers from **vanishing gradients** (i.e., gradients are very small), which makes NN **slow** to converge.
- ReLU leads to **rugged** decision boundaries, but converges **faster**.

Weight Initialization

Why Initialization Matters in Neural Networks?

- **Weight initialization** refers to choosing the initial values of the weights of the NN that sets the starting point for training.
- Then through optimization (e.g., SGD), the weights of the NN will get updated.
- Q: Can we set all the initial weights to zero?
- A: We should never set them to zero because it leads to the **symmetry problem**:
 - ▶ All neurons compute the same output because: Same weights $\rightarrow$ same dot product with input $\rightarrow$ same activation.
 - ▶ During backpropagation all neurons in the layer receive identical gradients, and they get updated in the same way.
 - ▶ Result is that the neurons remain **indistinguishable** from one another and learn the same features.
- The initial weights should be set to **small random values**. This is done for **breaking the symmetry**.

Why Initialization Matters in Neural Networks?

- If the weights are **too small** at initialization then they can cause:
 - ▶ **Vanishing gradients:** When gradients become very small as they are backpropagated through many layers, often approaching zero.
 - ▶ Recall that backprop involves multiplying gradients with activations (or weights). Small weights $\rightarrow$ small activations $\rightarrow$ small gradients.
 - ▶ The NN fails to learn because no gradients = no learning!
- If the weights are **too large** at initialization then they can cause:
 - ▶ **Exploding gradients:** When gradients become very large as they are propagated backward.
 - ▶ Same logic holds here, except the weights are too large, outputs and gradients can grow exponentially across layers.
 - ▶ This leads to numerical instability and divergence during training.

Popular Initialization Techniques

- A proper initialization method should avoid reducing or magnifying the magnitudes of input signals exponentially.
- **Xavier Weight Initialization**: Initialize weights so that the **variance of activations stays the same across layers**. Used in NN with sigmoid/tanh activation functions.

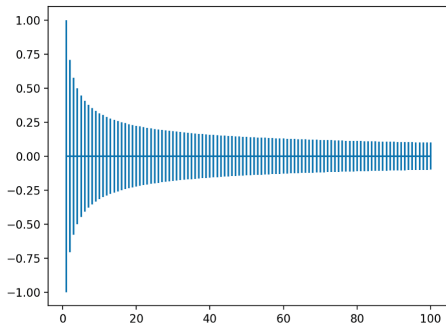
$$w \sim U\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right),$$

where n_{in} and n_{out} are the number of input and output nodes in a layer.

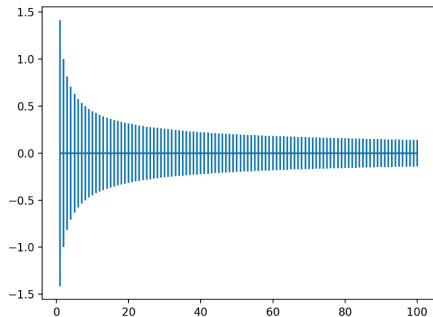
- **He Initialization**: Optimized for ReLU and its variants.

$$w \sim U\left(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{out}}}\right)$$

Popular Initialization Techniques



Xavier initialization



He initialization

- With very few input neurons (x-axis), the range in the weights is large (y-axis).
- The range rapidly drops with an increase in the number of neurons, after which it remains reasonably constant.

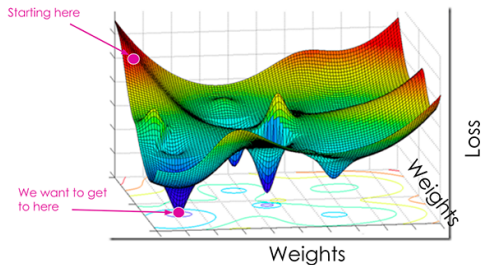
Optimization Algorithms

What is the Role of Optimization?

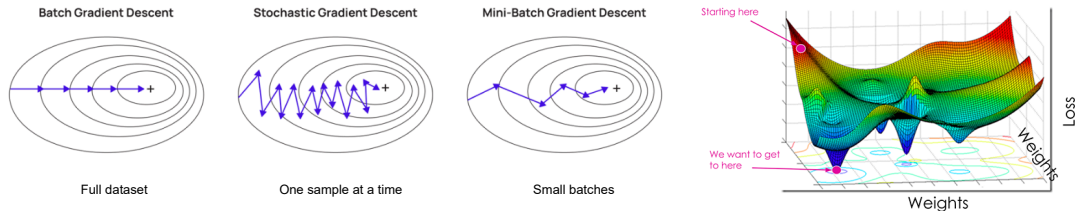
- **Goal:** Minimize the loss function by updating weights iteratively.
- Done using Gradient Descent optimization algorithm (or its variants like Stochastic Gradient Descent).
- Gradient Descent weight update rule:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} J,$$

where $\mathbf{w}$ are the weights, α is the learning rate, and $\nabla_{\mathbf{w}} J$ is the gradient of the loss function with respect to the parameters of the NN.



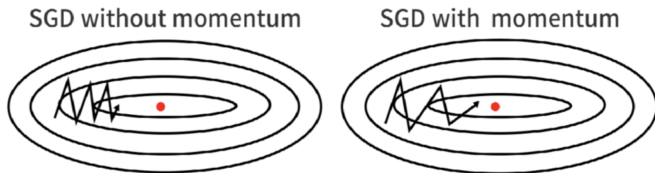
What is the Role of Optimization?



Algorithm	Description	Pros	Cons
Batch Gradient Descent	Model parameters are updated after processing the entire training dataset	Stable and accurate	Slow, requires lot of memory, can get stuck in local minima
Stochastic GD	Parameters are updated after processing one sample	Faster and can escape local minima	Noisy and updates are less accurate
Mini-batch SGD	Parameters are updated after processing a mini-batch of samples (say 64)	Best of both worlds. Less prone to oscillations compared to SGD, and faster than GD.	-

- Note, all modern-day NN are trained with mini-batch SGD.

SGD with Momentum



- A major issue of SGD is that it can oscillate if the loss landscape is narrow. Hence, **slow convergence** in deep valleys.
- For the Figure on the left, we want to learn slower in the vertical direction, and faster in the horizontal direction.
- **Momentum idea:** look at the past few updates and the average of their directions, and move in that direction.

SGD with Momentum

- More concretely, it adds a **velocity term** (or inertia) to build up speed in consistent directions.
- The weight update rule for SGD with momentum:

$$\mathbf{v}_t \leftarrow \underbrace{\beta \mathbf{v}_t + (1 - \beta) \alpha \nabla_{\mathbf{w}} J}_{\text{exponential moving average}}$$
$$\mathbf{w} \leftarrow \mathbf{w} - \mathbf{v}_t,$$

where $\mathbf{v}_t$ is the velocity at iteration t , and β is the momentum coefficient (usually set to 0.9).

- Intuitively, $\mathbf{v}_t$ captures the **dominant direction** of the past updates since it is a weighted average of 90% past gradients and 10% current gradient.
- Check here¹ for a nice visualization.

¹https://cdn-images-1.medium.com/v2/resize:fit:1000/1*Nb39bHHUWGXqgisr2WcLGQ.gif

Comparison of Different Optimizers

Optimizer	Key idea	Update rule (simplified)	Comments
SGD	Update weights using gradient from mini-batch	$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} J$	Slow convergence
SGD w/ momentum	Add inertia to the updates	$\mathbf{v}_t \leftarrow \beta \mathbf{v}_t + (1 - \beta) \alpha \nabla_{\mathbf{w}} J$	Faster convergence
Adagrad	Adjusts the learning rate for each parameter based on the accumulated sum of squared gradients G_t .	$\alpha \leftarrow \frac{\alpha}{\sqrt{G_t + \epsilon}}, G_t = (\nabla_{\mathbf{w}} J)^2$	No need to tune learning rate, as it decays over time
RMSProp	Exponentially decaying average of the squared gradients instead of accumulating the sum.	$G_t \leftarrow \beta G_{t-1} + (1 - \beta) G_t$	Prevents the learning rate from decreasing too quickly
Adam	Combines Momentum + RMSProp	$\mathbf{v}_t \leftarrow \beta_1 \mathbf{v}_t + (1 - \beta_1) \nabla_{\mathbf{w}} J$ $\mathbf{g}_t \leftarrow \beta_2 \mathbf{g}_t + (1 - \beta_2) (\nabla_{\mathbf{w}} J)^2$	Fast, adaptive, low tuning required

Loss Functions

What is the Role of Loss Functions?

- Loss function measures the difference between **predicted output** and **true target**.
- It quantifies “how wrong” the model is.
- The loss guides gradient descent and therefore learning.
- A well-chosen loss function aligns with the task objective (classification, regression, etc.).

Key insight: The choice of loss function defines what “good” performance means for your task.

Loss Functions for Classification

- **Goal:** Measure how well **predicted probabilities** match actual **class labels**.
- Two commonly used loss functions for classification:
 - ▶ Binary Cross-Entropy Loss (BCE) for binary classification
 - ▶ Cross-Entropy Loss (CE) for multiclass classification
- **Binary Cross-Entropy Loss:** Used for binary classification problems.

$$\mathcal{L}_{\text{BCE}} = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y})),$$

where y is the ground-truth label of the sample (either 0 or 1), and $\hat{y} \in [0, 1]$ is the prediction of the model.

- The above is the BCE loss for one sample. To compute the BCE loss for a mini-batch of size N , we take an average over the N samples:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N (y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})).$$

Loss Functions for Classification

- **Cross-Entropy Loss**: Used for multiclass classification problems.

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^K y_k \log \hat{y}_k,$$

where K is the number of classes, y_k is the ground-truth label corresponding to the k -th class and $\hat{y}_k$ is the predicted probability of the k -th class.

- The above is the CE loss for one sample. To compute the CE loss for a mini-batch of size N , we take an average over the N samples:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}.$$

Loss Functions for Regression

- **Goal:** Measure the difference between predicted and true **numerical values**.
- **Mean Squared Error (MSE) Loss:** Penalizes large errors (from outliers).

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2$$

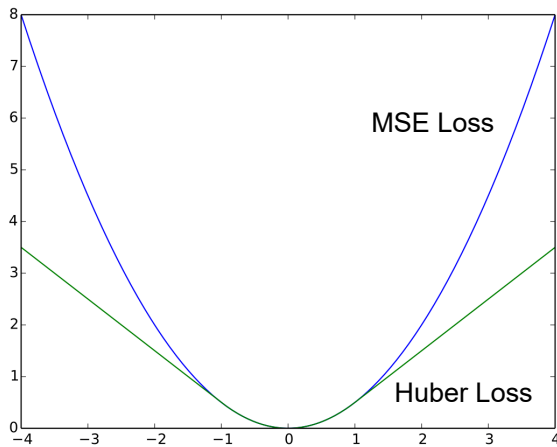
- **Mean Absolute Error (MAE) Loss:** Treats all errors equally.

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |y^{(i)} - \hat{y}^{(i)}|$$

- **Huber Loss:** Combines MSE (for small errors) and MAE (for large errors). Let $a = (y - \hat{y})$, and δ be a threshold.

$$\mathcal{L}_{\text{Huber}} = \begin{cases} \frac{1}{2}a^2 & \text{if } |a| \leq \delta \text{ (i.e., small errors)} \\ \delta(|a| - \frac{1}{2}\delta) & \text{otherwise (i.e., large errors)} \end{cases}$$

Loss Functions for Regression



Regularization Techniques

Why Regularization Matters in Neural Networks?

- Neural networks are flexible, but that makes them prone to **overfitting** – learning the noise and outliers, not the signal.
- Regularization helps models **generalize better** to unseen data.
- Common techniques for regularizing NN:
 - ▶ L1/L2 regularization
 - ▶ **Weight decay** (today)
 - ▶ **Dropout** (today)
 - ▶ Data augmentation
 - ▶ Early stopping

Weight Decay Regularization

- A technique that **shrinks weights** during training, discouraging large parameter values.
- Implemented by adding a **penalty** to the loss that increases with the square of the weights.

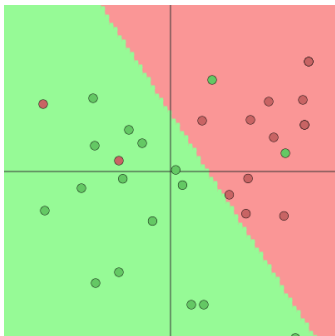
$$J_{\text{reg}}(\mathbf{w}) = \underbrace{J(\mathbf{x}, \mathbf{w})}_{\text{loss on training data}} + \underbrace{\frac{\lambda}{2} \sum_i w_i^2}_{\text{regularization}},$$

where λ controls the strength of the regularization term.

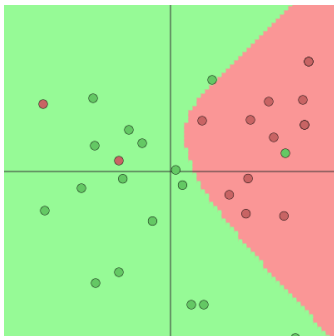
- Has the same impact as the ℓ_2 regularization, but in practice is incorporated inside the optimizer. Meaning no need to explicitly add $\sum_i w_i^2$ to the loss function. Simply specify its strength in the optimizer.
- Promotes **smoother and simpler** decision boundaries.

Impact of Weight Decay Regularization

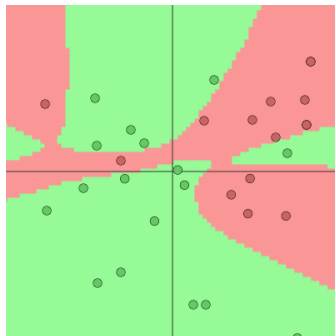
weight decay = 1



weight decay = 0.1



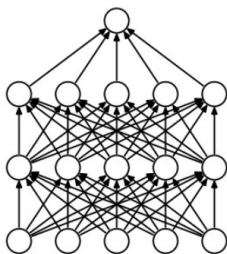
weight decay = 0.0



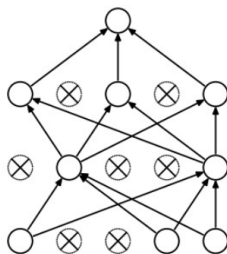
- Higher weight decay results in simpler and smoother decision boundary (left).
- No weight decay allows the NN to overfit to the outliers, resulting in complex decision boundary.

Dropout as Regularization

- Dropout randomly **deactivates neurons** during training.



(a) Standard Neural Net

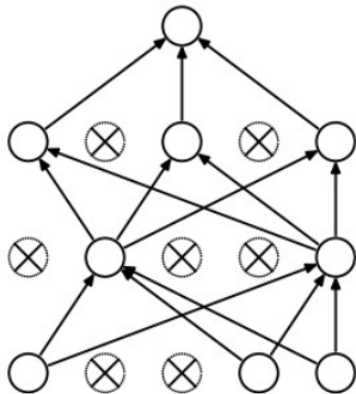


(b) After applying dropout.

- Dropout is implemented by only keeping a neuron active with some probability p (called the *keep probability*, a hyperparameter) during training, or setting it to zero with probability $(1 - p)$. Done independently for each training sample.
- Dropout is disabled at test time.

Understanding Dropout

- The intuition behind dropout is to prevent neural networks from becoming **overly reliant on specific neurons**.
- In other words, not rely on **any one pathway**, encouraging redundancy and robustness.
- It can also be seen as: dropout promotes the weights to be spread across the input neurons, and not make a certain weight too large in a given layer.



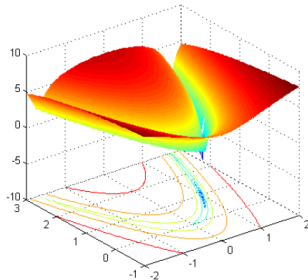
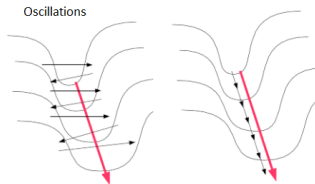
Inverted Dropout Pseudocode

```
1 def forward_pass(x, W, b, activation, p_keep, training):
2     '''
3     W, b:      weight matrix and bias vector of a layer
4     x:         inputs
5     activation: non-linear function (e.g., ReLU)
6     p_keep:    keep probability (e.g., 0.8 means keep 80% of units)
7     training:  boolean flag indicating whether in training mode
8     '''
9     z = W * x + b          # Linear transformation
10
11     if training:
12         mask = np.random.rand(z.shape) < p_keep
13         # an element is True if the random
14         # number is more than p_keep or False
15         # otherwise
16
17         z = z * mask        # Apply dropout mask
18         z = z / p_keep      # Inverted scaling
19
20     a = activation(z)       # Apply activation function
21
22     return a
```


Normalization Techniques

Why Normalization Matters in Neural Networks?

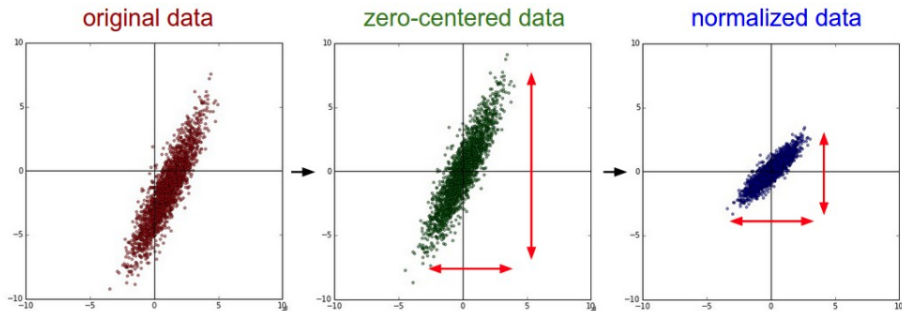
- It can happen that different input features are of different scales.
- If one feature is much larger scale than others, then it leads to very narrow and deep valleys, causing the optimization to oscillate, i.e., slow convergence.



- Other side effects include learning scale-sensitive patterns, i.e., overfitting.

Why Normalization Matters in Neural Networks?

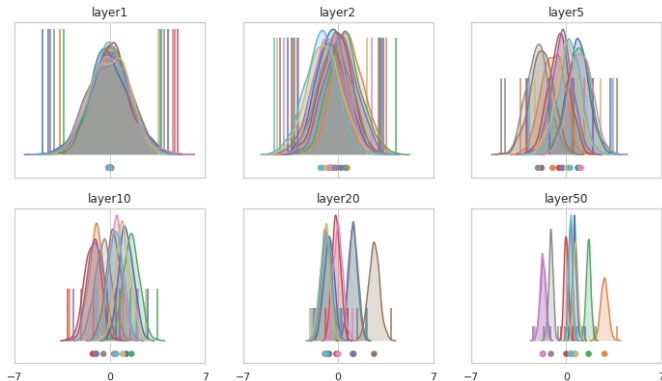
- **Normalization** refers to normalizing the data dimensions so that they are of approximately the same scale.
- One common technique: subtract the mean (i.e., zero-centering) and divide by the standard deviation. Now both the features have similar scales.



- As an effect, the loss landscape is no longer narrow and deep, ensuring smoother optimization.

Internal Covariate Shift

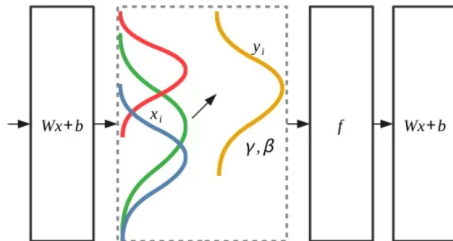
- In deep networks, input distributions vary widely across layers, it causes **shifting activations**, making training unstable.



- Each color represents the distribution of activations of a different neuron. We can observe that as we go deeper, the distributions start varying a lot.

Batch Normalization

- It is a technique for overcoming internal covariate shifty by **standardizing** the inputs to a layer for each mini-batch.



- It works as follows:
 - For a given mini-batch, compute mean μ and variance σ^2 of the intermediate features x .
 - Normalize each feature:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

- Scale (γ) and shift (β), which are learnable parameters:

$$y_i = \gamma \hat{x}_i + \beta$$

Summary & Takeaways

- No single “best” choice – everything depends on:
 - ▶ Data
 - ▶ The type of the task
 - ▶ Architecture depth
- NN training requires a lot of baby-sitting. Use validation set to find the optimal training configurations.
- Watch out for overfitting and unstable training.
- Neural networks are like recipes – ingredients matter, but how you combine and tune them is what makes them work.

References and Links

- For additional reading materials refer to the following links
 - ▶ [“Dive into Deep Learning”](#)². Chapter: 5
 - ▶ [“CS 229 Lecture Notes”](#)³. Chapter: 7
 - ▶ [“CS231n Lecture Notes”](#)⁴.

²https://d2l.ai/chapter_multilayer-perceptrons/index.html

³https://cs229.stanford.edu/main_notes.pdf

⁴<https://cs231n.github.io/>